



貴州大學

本科毕业论文（设计）

论文（设计）题目：融合威胁情报与大语言模型的网
络入侵检测系统设计

学 院：计算机科学与技术学院
专 业：信息安全
班 级：信息 2101
学 号：2100170012
学生姓名：田文静
指导教师：郭 春

2025 年 5 月 26 日

贵州大学本科毕业论文（设计）

诚信责任书

本人郑重声明：本人所呈交的毕业论文（设计），是在导师的指导下独立进行研究所完成。毕业论文（设计）中凡引用他人已经发表或未发表的成果、数据、观点等，均已明确注明出处。

特此声明。

论文（设计）作者签名：___田文静___

日 期：___2025年6月11日___



目录

摘要	IV
Abstract.....	V
第一章 绪论.....	1
1.1 研究的背景和意义	1
1.2 入侵检测技术国内外发展现状	1
1.2.1 国内外发展现状	1
1.2.2 发展趋势	3
1.3 主要研究内容	4
1.4 论文组成结构	5
第二章 入侵检测与大语言模型技术概述	6
2.1 入侵检测技术概述	6
2.1.1 Snort 规则语法	6
2.1.2 流量监控与告警机制	6
2.2 大语言模型在网络安全入侵检测中的应用	7
2.2.1 大语言模型概述	7
2.2.2 威胁情报解析与规则生成	7
2.2.3 大语言模型告警研判与误报分析	8
2.3 系统架构和开发技术	8
2.4 本章小结	8
第三章 系统需求分析	9
3.1 业务需求	9
3.1.1 系统核心功能描述	9
3.1.2 业务流程图	9
3.2 功能需求	9
3.2.1 功能需求概述	9
3.2.2 系统数据流图	10
3.3 非功能需求	10
3.3.1 安全需求	10



3.3.2 性能需求	10
3.3.3 运行环境需求	11
3.4 系统逻辑模型	11
3.5 本章小结	11
第四章 系统设计	12
4.1 系统总体设计	12
4.1.1 设计原则	12
4.1.2 设计目标	12
4.1.3 系统工作流程	12
4.2 系统功能设计	13
4.2.1 服务端功能	13
4.2.2 系统功能模块划分	13
4.3 系统功能模块设计	14
4.4 数据库设计	14
4.4.1 数据库概念结构设计	14
4.4.2 数据库逻辑结构设计	16
4.5 本章小结	21
第五章 系统实现	22
5.1 系统开发工具	22
5.2 威胁情报处理模块的实现	22
5.3 流量监控与告警模块的实现	24
5.4 大模型告警研判模块的实现	25
5.4.1 告警数据提取与大模型分析流程	25
5.4.2 研判结果展示与误报优化	26
5.5 黑名单反制功能实现	27
5.6 本章小结	28
第六章 系统测试与分析	29
6.1 系统测试环境	29
6.2 测试数据集来源与构成	29



6.3 功能测试	30
6.3.1 生成规则有效性测试	30
6.3.2 告警研判响应时间测试	31
6.3.3 系统基础功能测试	31
6.4 性能测试	32
6.5 本章小结	32
第七章 总结与展望	33
7.1 总结	33
7.2 展望	34
参考文献	35
致谢	36



融合威胁情报与大语言模型的网络入侵检测系统设计

摘要

随着互联网技术的飞速发展，网络攻击手段愈发复杂多样，网络入侵事件频繁发生，给个人、企业乃至国家网络安全带来了严峻挑战。传统网络入侵检测系统（Network intrusion detection system, NIDS）在面对新型攻击时，因规则更新滞后、检测准确率不足等问题，难以有效应对，无法满足当前网络安全防护的迫切需求。因此，提升 NIDS 的智能化水平和实时响应能力成为网络安全领域亟待解决的关键问题。当前，尽管已有众多研究致力于改进 NIDS，但仍存在明显不足。一方面，传统方法对系统防御单点打孔，难以满足当前系统测试的高标准；另一方面，测试过程中未加限制的漏洞挖掘和攻击可能影响目标系统，甚至导致服务器崩溃和宕机。同时，现有系统在融合先进技术与提升综合性能方面仍有较大提升空间。

针对上述问题，本文提出了一种融合威胁情报与大语言模型的网络入侵检测系统设计。该方案创新性地将先进的自然语言处理技术与传统入侵检测工具相结合，通过自动化规则生成与高效告警研判机制，显著提升 NIDS 的检测效率和准确性。基于 Django 框架，详细设计了系统的整体架构，并完成了系统的实现与测试。通过实验结果表明，所提出的融合方案能够有效提升入侵检测系统（IDS）的检测性能，在检测效率和准确性方面均有显著改善，为网络安全防护提供了更为可靠的技术支持。

关键词：网络入侵检测系统，Snort，威胁情报，大语言模型



Design of Network Intrusion Detection System Integrating Threat Intelligence and Large Language Model

Abstract

With the rapid development of Internet technology, network attacks have become more complex and diverse, and network intrusions occur frequently, posing serious challenges to the network security of individuals, enterprises and even countries. Traditional network intrusion detection systems (NIDS) are unable to effectively respond to new types of attacks due to issues such as delayed rule updates and insufficient detection accuracy, and cannot meet the urgent needs of current network security protection. Therefore, improving the intelligence level and real-time response capability of NIDS has become a key issue that urgently needs to be addressed in the field of network security. Currently, despite numerous studies dedicated to improving NIDS, there are still significant shortcomings. On the one hand, traditional methods for single point punching in system defense are difficult to meet the high standards of current system testing; On the other hand, unrestricted vulnerability mining and attacks during the testing process may affect the target system and even lead to server crashes and downtime. At the same time, there is still significant room for improvement in the integration of advanced technologies and the enhancement of overall performance in existing systems.

In response to the above issues, this article proposes a design scheme for a network intrusion detection system that integrates threat intelligence and big language models. This solution innovatively combines advanced natural language processing technology with traditional intrusion detection tools, significantly improving the detection efficiency and accuracy of NIDS through automated rule generation and efficient alarm analysis mechanisms. Based on the Django framework, the overall architecture of the system was designed in detail, and the implementation and testing of the system were completed. The experimental results show that the proposed fusion scheme can effectively improve the detection performance of intrusion detection systems (IDS), with significant improvements in detection efficiency and accuracy, providing more reliable technical support for network security protection.



Key words: Network intrusion detection system, Snort, Threat intelligence, Large Language Model (LLM)



第一章 绪论

1.1 研究的背景和意义

在当今数字化浪潮的推动下，互联网已经如同空气和水一般，深入渗透到人们生活的方方面面，成为现代社会不可或缺的基础设施。从日常的社交娱乐、在线购物，到企业的运营管理、政府的服务提供，互联网的应用无处不在，极大地提高了人们的生活效率和社会运行水平。然而，互联网的广泛普及和深入应用也带来了一系列严峻的网络安全问题。

网络入侵行为日益猖獗，其危害范围广泛且影响深远。对于个人而言，网络入侵可能导致个人隐私信息泄露，如身份证号码、银行卡号、家庭住址等，进而引发诈骗、盗刷等财产损失，甚至威胁到个人的人身安全。对于企业来说，网络入侵可能造成商业机密泄露、客户数据被盗取，导致企业声誉受损、市场份额下降，严重时甚至可能导致企业破产倒闭。而对于国家安全和社会稳定，网络入侵行为更是可能成为别有用心之人进行破坏、窃取国家机密、扰乱社会秩序的工具，对国家的政治、经济、军事等领域造成难以估量的损失。

但是在传统的网络入侵检测系统在面对日益复杂多变的网络环境时，逐渐暴露出诸多问题。规则更新滞后使得系统无法及时应对新型的网络攻击手段，误报率高则让安全管理人员疲于应对大量的虚假告警，分散了精力，降低了工作效率。这些问题严重制约了传统入侵检测系统在实际应用中的效果，难以满足当前复杂多变的网络环境需求。

因此，研究基于大模型和威胁情报的网络入侵检测系统具有重要的现实意义^[1]。大模型具有强大的学习能力和数据处理能力，能够对海量的网络数据进行分析和挖掘，自动发现潜在的入侵模式；威胁情报则提供了实时的、全面的网络攻击信息，帮助系统更准确地判断入侵行为的性质和危害程度^[2]。通过将大模型和威胁情报相结合，有望构建一个更加智能、高效、准确的网络入侵检测系统，为网络安全提供更有力的保障。

1.2 入侵检测技术国内外发展现状

1.2.1 国内外发展现状

近入侵检测技术自 20 世纪 80 年代诞生以来，经历了多个阶段的发展。最初是基于规则的误用检测，通过预先定义好的攻击规则来识别已知的入侵行为。这种方法简单直接，但对于未知的攻击手段则无能为力。随着技术的发展，逐渐出现了基于行为的异



常检测，它通过建立正常行为的模型，将与正常行为偏差较大的活动视为潜在的入侵行为。

入侵检测技术自 20 世纪 80 年代诞生以来，历经了多个发展阶段。起初，基于规则的误用检测占据主导地位，它通过预先设定好的攻击规则来识别已知的入侵行为，这种方式简单直接，能有效应对常见的、已知的攻击场景。然而，随着网络环境日益复杂，新型攻击手段层出不穷，基于规则的检测方法暴露出了明显的局限性，面对未知攻击时显得力不从心。为了弥补这一缺陷，基于行为的异常检测技术应运而生。该技术通过构建正常行为模型，将与正常行为模式偏差较大的活动识别为潜在的入侵行为，大大提升了对未知攻击的检测能力^[3]。

在国际学术界，众多学者在入侵检测领域深入钻研，取得了诸多开创性成果。以 Snort 为代表的开源网络入侵检测系统，凭借其开源性、灵活性以及可扩展性等优势，在全球范围内得到了广泛应用，为众多网络安全防护体系提供了坚实的基础。近年来，机器学习与威胁情报的融合成为研究热点。卡内基梅隆大学团队依托 KDD Cup 1999(Knowledge Discovery and Data Mining Cup)数据集，运用决策树、随机森林等算法，实现了对网络攻击的高效分类。实验结果显示，随机森林模型在加拿大网络安全研究所 2017 年入侵检测系统数据集上的准确率高达 99.54%。此外，麻省理工学院计算机科学与人工智能实验室开发的 AI2 系统，通过动态学习网络行为模式，将误报率降低了 5 倍，检测效率提升 3 倍，为入侵检测技术的智能化发展开辟了新的路径。近期，国外不少科研团队聚焦于深度学习在入侵检测中的深度应用。部分研究机构借助卷积神经网络和递归神经网络对大规模网络流量数据展开分析，成功实现对多种复杂网络攻击的有效检测。像是在车联网领域，有研究利用自然语言处理技术对原始流量进行令牌化处理，并在卷积神经网络（Convolutional Neural Network,CNN）和双向长短期记忆架构中融入注意力机制，在车内网络场景的 Car - Hacking 数据集上达到了 100%的准确率，车外网络场景使用 CICIoT2023 数据集时，准确率也高达 98%，极大提升了入侵检测的精准度。

国内在入侵检测技术方面同样成绩斐然。清华大学段海新团队长期深耕网络安全领域，在基础协议安全研究方面成果卓著。他们发现了 DNS、HTTPS 等基础协议的安全漏洞，相关成果被 IETF(Internet Engineering Task Force) 等国际标准化组织采纳，有力地推动了网络安全领域的国际合作与发展。奇安信“天眼”系统整合了威胁情报、文件虚拟执行和机器学习技术，在应对高级威胁方面表现出色，成功检测出永恒之蓝、高



级可持续攻击等复杂攻击手段，在金融、政府等关键行业累计处置超百起重大安全事件，市场份额连续三年位居国内第一。中科院夏葳团队提出的基于联邦学习和原型学习的物联网入侵检测框架，针对资源受限环境下的安全挑战，实现了高效的恶意行为识别，相关成果发表于 IEEE ICPADS 2023，为物联网安全防护提供了新的解决方案。西安邮电大学贾阳博士团队提出的深度神经网络检测方法，在入侵检测准确率上取得了重大突破，该成果发表于 IET Information Security 期刊，并荣获 2021 年度最佳论文奖，准确率高达 99.9%，展现了国内在该领域的前沿研究水平。江苏智联联合西交利物浦大学、清华大学苏州汽车研究院于 2024 年 9 月发布智能网联汽车入侵检测研究新成果，其提出的结合机器学习和布隆过滤器技术的入侵检测系统，在 Car - Hacking 数据集上准确率达 99.99%，误报率仅 0.0002%；在 CIC - IDS - 2017 数据集上，准确率达 99.92%，误报率降至 0.008%，还大幅降低了计算资源消耗与检测延迟，满足了车载环境实时性与轻量化要求。并且，2025 年 1 月，西安热工研究院有限公司申请“一种工业控制系统入侵检测方法及相关装置”专利，通过采集设备状态数据和网络流量数据，经预处理后分别输入对应异常检测模型，有效解决现有单一入侵检测技术误报和漏报问题。

尽管国内外在入侵检测技术上已取得显著进展，但网络攻击手段的不断演进使得传统检测技术面临严峻挑战。新型网络攻击愈发隐蔽、多样且变化迅速，例如域名前置攻击，利用吞吐量波动巧妙规避传统检测方法的监测，使得入侵检测系统难以在第一时间发现并准确识别此类攻击，给网络安全防护带来了巨大压力^[4]。

1.2.2 发展趋势

近年来，随着大模型技术的快速发展和威胁情报的广泛应用，入侵检测技术正逐步向智能化、自动化方向发展。通过结合大模型算法和威胁情报数据，网络入侵检测系统能够实现更精准的入侵行为识别和更高效的告警研判^[5]。例如，中国电信“见微”安全大模型通过处理海量告警数据，将云网告警压缩率提升至 99.2%，显著缓解了安全团队的“告警疲劳”问题。国际研究中，GPT-4 等大语言模型通过上下文学习技术，在入侵检测任务中准确率提 90%，F1(Formula 1 Score)分数超过 95%。此外，Google Research 提出的 Pathways 架构，通过稀疏激活混合专家网络，能够在处理大规模网络流量数据时，动态分配计算资源，大幅提升入侵检测效率，其在公开数据集上的实验表明，相比传统模型，检测速度提升了 3 倍以上。

大模型可以对海量的网络数据进行深度学习，自动提取特征和模式，提高检测的准



确性。例如，清华大学团队提出的 SDFUZZ (State-Driven Directed Fuzzer) 和 LABRADOR (Response-Guided Directed Fuzzing for Black-Box IoT Devices) 等模糊测试框架，通过动态生成测试用例，有效发现了物联网设备和 Web 应用防火墙的协议级规避漏洞，相关成果被 USENIX Security 录用。同时，麻省理工的研究人员开发出一种基于 Transformer 架构的入侵检测模型，该模型通过多头注意力机制捕捉网络流量中的长距离依赖关系，在面对新型加密攻击时，检测准确率较传统深度学习模型提高了 15%。威胁情报则可以为系统提供实时的攻击信息和背景知识，帮助系统更好地理解入侵行为的意图和危害。奇安信威胁情报中心监测到针对中国的 47 个 APT(Advanced Persistent Threat) 组织，并通过 ATT&CK(Adversarial Tactics, Techniques, and Common Knowledge) 框架实现攻击链分析，为防御提供了关键支撑。而微软的 Microsoft Threat Intelligence 平台，通过整合全球范围内的威胁情报数据，构建了动态的攻击行为图谱，能够提前预测潜在的攻击趋势，为企业用户提供前瞻性的安全防护建议。

未来，入侵检测技术将更加注重实时性、准确性和可扩展性等方面的提升，以适应不断变化的网络环境。例如，联邦学习技术被应用于分布式入侵检测^[6]，在保护数据隐私的同时实现跨域协同检测。IBM(International Business Machines Corporation)推出的基于联邦学习的区块链入侵检测方案，通过在多个节点间共享加密的模型参数而非原始数据，在金融行业的应用中，成功实现了跨机构的攻击信息共享，检测效率提升了 40%。此外，结合协议分析和行为分析的新型检测技术，如奇安信“天眼”系统的全包取证和线索可视化图谱分析，能够完整呈现攻击路径，提升回溯和响应能力。

1.3 主要内容

(1)设计融合大模型与威胁情报的入侵检测系统架构

为解决上述问题，本研究提出融合大语言模型与威胁情报的网络入侵检测系统设计方案。利用大语言模型强大的自然语言处理能力，对非结构化威胁情报进行深度解析，将其转化为可用于检测的结构化信息，并自动生成 Snort 规则，提升规则生成的及时性与准确性。同时，构建多维度告警研判机制，综合考虑攻击频率、来源可信度、影响范围等因素，降低误报率。此外，设计基于 iptables 的恶意 IP 自动封禁功能，实现对威胁的快速阻断。在系统架构上，基于 Django 框架进行分层设计，划分威胁情报处理、流量监控、告警分析及可视化反制等多个功能模块，确保系统结构清晰、易于扩展与维护。

(2)实现并测试该网络入侵检测系统原型



基于上述设计，开展系统的实现与测试工作。通过集成 Snort 流量引擎，实现对网络流量的实时监测与初步分析；调用 Qwen3 大模型 API，完成威胁情报解析与规则生成功能。利用 Django 框架将各个功能模块进行整合，开发出网络入侵检测系统原型。在系统实现后，进行全面的测试与性能测试，验证威胁情报处理的准确性、告警分析的可靠性以及恶意 IP 封禁的及时性等。通过模拟多种网络攻击场景，对系统的检测能力与响应速度进行压力测试，不断优化系统性能，确保系统能够在实际网络环境中稳定、高效运行，实现提升入侵检测智能化与实时性的研究目标。

1.4 ~~论文组成结构~~

本文共分为七章。第一章为绪论，介绍研究背景和意义、国内外发展现状与趋势以及主要研究内容。第二章为入侵检测与大模型技术分析，概述入侵检测技术原理和大模型在网络安全中的应用；第三章为系统需求分析，详细分析系统业务需求、功能需求和非功能需求。第四章为系统设计，设计系统总体架构、功能模块和数据库结构；第五章为系统实现，介绍系统部署方式及各模块的实现过程。第六章为系统测试与分析，对系统进行功能测试和性能测试。第七章为总结与展望，总结研究成果并展望未来发展方向。

第二章 入侵检测与大语言模型技术概述

2.1 入侵检测技术概述

2.1.1 Snort 规则语法

目前的网络入侵检测系统，如 Snort，都是使用特定的规则语法来定义如何检测潜在的恶意活动。这些规则是 NIDS 的核心，允许用户描述应视为可疑或恶意的具体网络行为模式。例如 snort 的规则组成如图 2.1 所示。

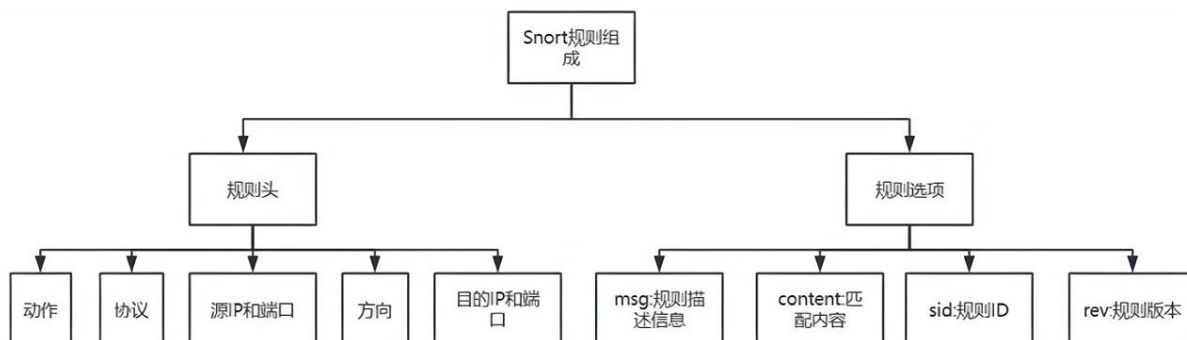


图 2.1 Snort 规则组成解释图

Snort 规则头包含了匹配数据包所需的基本信息，包括动作、协议、源 IP 地址、源端口、方向指示符、目的 IP 地址、目的端口。

2.1.2 流量监控与告警机制

传统网络入侵检测系统的流量监控与告警机制依托多层技术链路实现对网络行为的实时监测与异常响应，其核心流程可拆解为数据捕获、协议解析、规则匹配、告警生成四个关键环节：

在流量捕获层面，系统基于 libpcap 开源库实现原始数据包抓取，通过混杂模式监听网卡（如 eth0）获取本网段内所有流量。以 Snort 为例，通过命令 `sudo snort -i eth0 -c /etc/snort/snort.conf` 启动监听，实时采集双向流量数据，并通过 `pcap_dispatch` 函数实现数据包的高效捕获与预处理。

协议解析环节采用分层处理策略：首先解析链路层协议，其次提取网络层信息和传输层字段，最终剥离应用层载荷。例如，对 TCP 数据包，系统会解析序列号与窗口大小，为后续规则匹配提供时序与流量控制维度的元数据。

告警生成与输出包含标准化字段：时间戳、规则 ID（如 127268）、威胁等级（低/中



/高)、源/目 IP/端口、协议类型及载荷片段（如图 5.5 前端展示的"Potential Nmap TCP Connect Scan"告警）。告警数据可通过 barnyard2 插件同步至 MySQL 数据库（如 detection_alertevent 表），或写入 snort.log 文件，供管理员通过 Web 界面筛选查询（如按源 IP 统计攻击频率）。

传统机制的局限性体现在：被动告警需人工处置，响应滞后；单一告警缺乏攻击链时序关联（如端口扫描→漏洞利用），误报率高达 65%以上；跨设备协同能力不足，难以识别分布式攻击。论文后续提出的大模型研判模块（5.4 节）与自动化反制功能（5.5 节），正是针对这些痛点，通过多维度数据关联与动态防御策略，提升检测准确性与响应效率。

2.2 大语言模型在网络安全入侵检测中的应用

2.2.1 大语言模型概述

大语言模型（Large Language Model, LLM）是一种基于深度学习技术的人工智能模型，通过在海量文本数据上进行无监督预训练，学习语言的语法、语义和语用规则，从而具备强大的自然语言处理能力。这些模型能够理解输入文本的含义，并生成连贯、准确且符合语境的文本输出。以 GPT-4、Qwen3 等为代表的大语言模型，在多种自然语言处理任务中展现出了卓越性能，如文本生成、问答系统、机器翻译等。其核心优势在于对上下文的深度理解以及泛化能力，能够处理多样且复杂的语言场景。本系统选择 Qwen3，因其强大的语义理解能力、对非结构化数据的处理优势，与阿里云 AI 云盾的安全防护体系形成技术协同，在网络安全告警分析上有着明显的优势。

2.2.2 威胁情报解析与规则生成

威胁情报通常以非结构化或半结构化的形式存在，如技术报告、博客文章、论坛讨论、漏洞公告等。大语言模型能够通过对这些文本进行自然语言处理，提取出关键的威胁实体和行为特征，包括恶意 IP、域名、攻击载荷、漏洞编号、攻击模式等。模型首先对输入情报进行语义解析，识别其中涉及的攻击类型和技术细节，并结合已有的知识库（如 ATT&CK 框架）进行上下文映射。随后，将提取的信息转化为标准化格式，为后续规则生成提供数据基础。在规则生成阶段，系统根据预设的规则模板将结构化情报信息自动填充到对应字段中，生成可部署的检测规则。通过引入反馈机制，系统可以根据实际检测效果不断优化生成策略，提升规则的准确性和泛化能力^[7]。



2.2.3 大语言模型告警研判与误报分析

在入侵检测系统运行过程中，会产生大量告警日志，其中包含大量重复、冗余或误报的事。大语言模型可以对原始告警信息进行多维度分析，包括告警来源、攻击特征、时间序列、目标资产等^[8]。模型通过学习历史告警样本中的真实攻击模式和误报特征，建立上下文感知的研判逻辑。在处理新告警时，模型综合当前网络环境信息、用户行为、系统日志等辅助数据，判断该告警是否构成真实威胁。同时，系统可根据误报特征自动分类，标记出常见的误报类型，如扫描误触发、合法流量误判等。输出结果包括告警等级、处置建议以及误报标识，供安全团队参考^[9]。

2.3 系统架构和开发技术

本系统在后端开发方面采用了 Django 框架，利用其强大的对象关系映射（Object Relational Mapping, ORM）功能和模板引擎来简化数据库操作与前端页面开发流程。Django 的灵活性和可扩展性为系统的快速构建和迭代提供了坚实基础。在前端展示层面，选用了 SimpleUI 作为 Django 的管理界面增强工具，通过其直观的用户界面和丰富的功能组件，极大地提升了用户体验和管理效率。为了有效捕获和分析网络威胁数据，同时采用了 Snort 作为网络入侵检测系统的捕获引擎。Snort 以其高度的灵活性和可扩展性，能够实时检测并响应各种网络攻击行为，为威胁情报的收集提供了有力支持。在数据存储方面，本系统设计了合理的数据库结构，并选用了 MySQL 作为后端数据库。MySQL 以其高性能、高可靠性和易用性，确保了威胁情报数据、规则数据和告警数据等信息的快速存储、检索和分析处理。此外，为了进一步提升系统的智能化水平，本系统还集成了 Qwen3 大模型 API 调用功能。通过调用 Qwen3 提供的强大自然语言处理和分析能力，系统能够更准确地理解用户意图，实现更精准的数据分析和决策支持。

2.4 本章小结

本章概述了入侵检测技术的原理和方法，并介绍了大模型在网络安全领域的应用前景。同时，本章还对相关技术进行了简要介绍和分析，为后续章节的系统设计和实现提供了理论基础和技术支持。

第三章 系统需求分析

3.1 业务需求

3.1.1 系统核心功能描述

本系统聚焦基于大模型与威胁情报的网络入侵检测，核心功能围绕威胁情报处理、规则生成和告警分析展开。在威胁情报处理上，需从安全厂商报告、漏洞库等多源获取数据，通过 API 等多种协议保障情报实时更新，利用大模型解析非结构化文本，提取关键威胁信息，并融合校验确保情报质量。规则生成模块要依据处理后的情报，结合模板与模型学习能力动态生成规则，根据历史数据和业务场景优化规则参数，同时提供可视化界面方便规则管理。告警及分析模块需实时监控网络流量，解析数据包识别入侵特征，借助大模型多维度分析告警，区分真实攻击与误报并分级分类，对严重告警自动触发阻断攻击源、隔离服务器等响应策略^[10]，实现多部门协同处置安全事件。

3.1.2 业务流程图

系统业务流程图 3.1 如下所示。

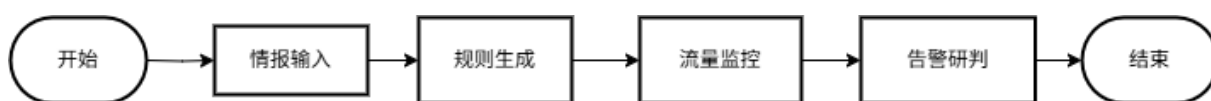


图 3.1 系统业务流程图

具体来说，系统首先从外部源获取威胁情报数据并进行解析和处理；然后根据处理后的威胁情报数据生成相应的入侵检测规则；接着对捕获到的网络流量进行实时监控和分析处理；最后根据预设规则对检测到的可疑行为进行告警研判并生成相应的告警信息。

3.2 功能需求

3.2.1 功能需求概述

系统需实现以“威胁情报驱动检测、大模型智能研判、可视化高效处置”为核心的功能体系，形成“情报-检测-研判-响应-管理”全链路闭环。具体要具备从开源平台、企业私有源及用户上传文件采集威胁情报^[11]，具体需具备从多源采集威胁情报，通过大模型解析提取 IOC 并生成检测规则的能力；拥有通过流量引擎监听网卡捕获流量，执行规则匹配与异常行为分析的功能；可融合多源数据，调用大模型分析告警上下文输出研判结果与处置建议并优化误报；能够呈现实时告警列表、趋势分析图表及规则管理界面，支持搜索、标记与批量处置；基于网络层防护机制实现恶意 IP 自动封禁与防御日志记

录；集成用户认证、数据存储备份及系统监控，利用邮件服务实现异常状态通知。各模块功能对应论文中图 3.2 数据流图与图 4.2 功能模块划分图。

3.2.2 系统数据流图

系统数据流图如图 3.2 所示。

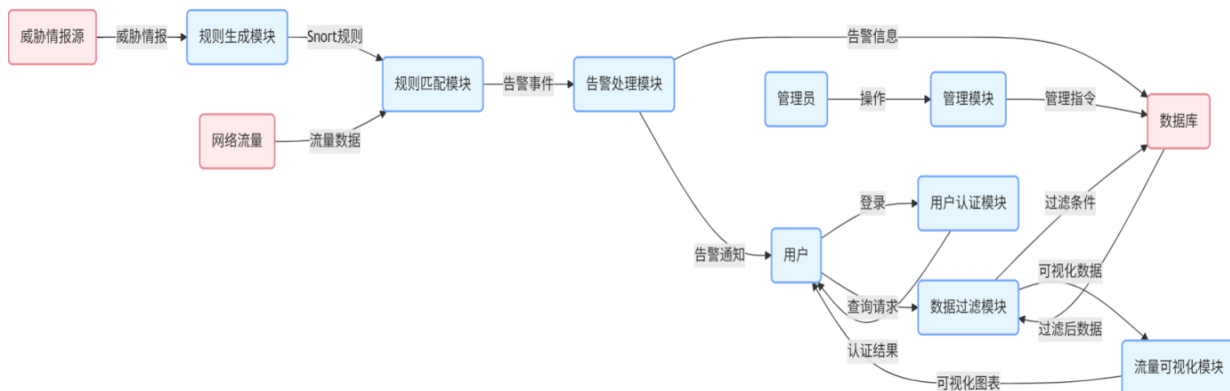


图 3.2 系统数据流图

威胁情报源提供威胁情报，经规则生成模块生成 Snort 规则。同时，网络流量数据也进入规则匹配模块，与 Snort 规则进行匹配，产生告警事件。告警事件被送入告警处理模块，该模块与管理模块交互，管理员通过用户认证模块登录系统进行操作，管理模块从数据库获取管理指令。告警处理模块将告警信息通知用户，用户可发起查询请求，经数据过滤模块处理后，从数据库获取可视化数据，最终由流量可视化模块生成可视化图表^[12]。

3.3 非功能需求

3.3.1 安全需求

系统安全需求主要包括规则准确性和系统稳定性两个方面。规则准确性要求系统能够准确识别并处理各种类型的入侵行为；系统稳定性则要求系统能够在长时间运行过程中保持稳定可靠的性能表现。

3.3.2 性能需求

系统性能需求主要包括实时告警响应和大模型分析延迟两个方面。实时告警响应要求系统能够在检测到可疑行为时立即触发告警机制并生成相应的告警信息；大模型分析延迟则要求系统能够在合理的时间范围内完成对告警数据的分析和研判工作。

3.3.3 运行环境需求

系统运行环境需求主要包括 Python/Django 开发环境、Snort 部署以及 PyCharm 等开发工具的支持。Python/Django 开发环境用于后端服务的开发和部署；Snort 部署用于捕获和分析网络流量；PyCharm 等开发工具则用于提高开发效率和代码质量。

3.4 系统逻辑模型

系统逻辑模型如图 3.3 所示。

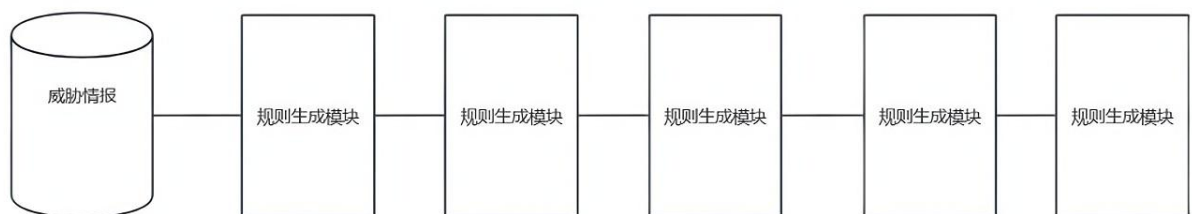


图 3.3 系统逻辑模型

(1)威胁情报源：作为系统的输入端，提供外部威胁情报信息，这些情报是后续规则生成的基础。

(2)规则生成模块：接收来自威胁情报源的数据，分析并生成相应的安全规则。

(3)流量监控模块：基于规则生成模块产生的规则，通过 Snort 对网络流量进行实时监控。

(4)告警及分析模块：对流量监控模块捕获的数据通过 Snort 检测并生成相应的告警信息。通过邮箱通知给用户，并利用大模型研判分析告警。

(5)可视化模块：将告警及分析模块生成的信息以直观的方式展示出来，通过图表、报表等形式，使用户对告警和攻击趋势有更清晰的认识。

(6)管理员界面：最终的输出端，供管理员查看和操作。管理员可以通过此界面获取系统的运行状态、处理告警信息等。

3.5 本章小结

本章对系统业务需求、功能需求和非功能需求进行了详细分析，并设计了系统业务流程图和数据流图。同时，本章还给出了系统逻辑模型以便更好地理解 and 描述系统各模块之间的交互关系和数据流向。

第四章 系统设计

4.1 系统总体设计

4.1.1 设计原则

系统总体设计遵循模块化和可扩展性原则。模块化原则要求系统各模块之间保持相对独立性和可重用性。可扩展性原则要求系统能够方便地扩展新功能和优化现有功能以满足不断变化的业务需求和技术发展趋势^[13]。

4.1.2 设计目标

系统设计目标主要包括自动化规则生成和高效告警研判两个方面。自动化规则生成要求系统能够根据威胁情报数据自动生成相应的入侵检测规则以减少人工干预和降低误报率。高效告警研判则要求系统能够在短时间内完成对告警数据的分析和研判工作以提高响应速度和准确性^[14]。

4.1.3 系统工作流程

系统工作流程图如图 4.1 所示。^

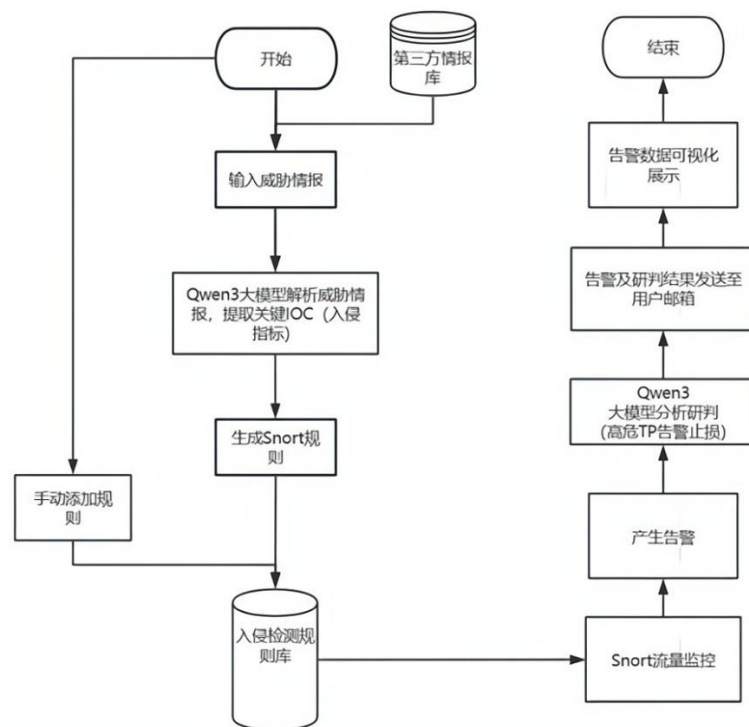


图 4.1 系统工作流程图

本系统的工作流程是从用户输入威胁情报开始，通过大模型分析提取关键 IOC，生成 Snort 规则并存入入侵检测规则库，进行流量监控、告警生成及研判，最终以可视化方式展示结果的完整安全监测与响应过程。

4.2 系统功能设计

4.2.1 服务端功能

服务端功能主要包括规则生成、流量监控、告警分析和黑名单反制四个部分。规则生成模块负责根据威胁情报数据生成相应的入侵检测规则；流量监控模块负责对捕获到的网络流量进行实时监控和分析处理；告警分析模块则负责对检测到的可疑行为进行告警研判并生成相应的告警信息，最后如果需要对攻击的源 IP 进行反制还可以将其加入到系统黑名单里面^[15]。

4.2.2 系统功能模块划分

功能模块划分如图 4.2 所示。

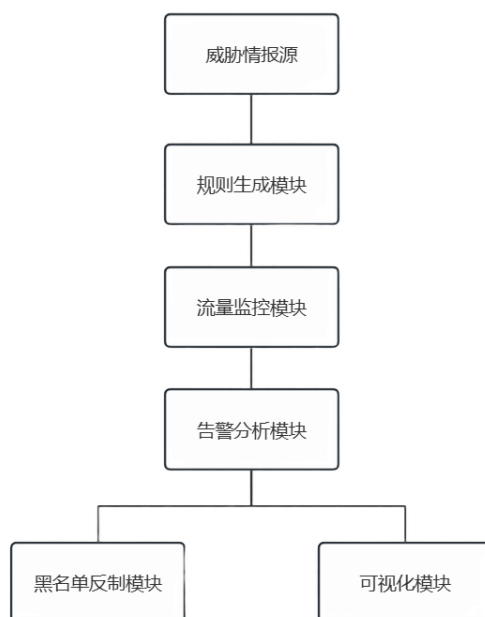


图 4.2 系统功能模块划分

在网络入侵检测系统中，威胁情报源为规则生成模块提供数据支持。规则生成模块依据威胁情报生成入侵检测规则，这些规则被传递给流量监控模块。流量监控模块利用规则对网络流量进行实时监控，一旦发现可疑流量，将相关数据发送给告警分析模块。告警分析模块对告警数据进行研判和处理，生成告警信息。同时，对于需要进行反制的



攻击源 IP，告警分析模块会将其信息传递给黑名单反制模块，实施封禁等操作。此外，告警分析模块生成的告警信息也会提供给可视化模块，以直观的方式展示给管理员，方便其查看和处理^[16]。

4.3 系统功能模块设计

(1)规则生成模块：负责从威胁情报源获取数据，利用大语言模型等技术进行解析和处理，生成符合 Snort 等网络入侵检测系统要求的规则。具体包括对威胁情报的提取、关键信息的识别以及规则的格式化生成等功能，确保生成的规则能够准确检测潜在的入侵行为。

(2)流量监控模块：基于规则生成模块产生的规则，使用 Snort 工具对网络流量进行实时捕获和分析。持续监测网络流量，将捕获到的流量数据与预设规则进行匹配，判断是否存在可疑行为，一旦发现匹配情况，及时将相关告警数据发送给告警分析模块。

(3)告警分析模块：接收流量监控模块传来的告警数据，进行深度分析和研判。利用大语言模型对告警数据进行处理，判断告警的真实性、攻击意图、威胁等级等，并生成详细的告警信息。同时，根据研判结果，决定是否需要将攻击源 IP 提交给黑名单反制模块进行处理。

(4)黑名单反制模块：根据告警分析模块的指令，对攻击源 IP 进行封禁等反制操作。通过自动化的 IP 封禁与解封机制，采用 iptables 等技术实现网络层的阻断。记录封禁和解封的相关信息，确保对恶意 IP 的有效管控，提高系统的安全性。

(5)可视化模块：将告警分析模块生成的告警信息以及相关的分析结果进行可视化展示。以直观的图表、报表等形式呈现给管理员，包括告警的时间分布、源 IP 和目标 IP 的统计、威胁等级的占比等信息，帮助管理员快速了解网络安全状况，及时做出决策。

4.4 数据库设计

4.4.1 数据库概念结构设计

(1)用户权限管理 E-R 图

系统的用户权限管理主要基于 Django-admin 框架所设计，其用户权限管理 E-R 图如图 4.3 所示。

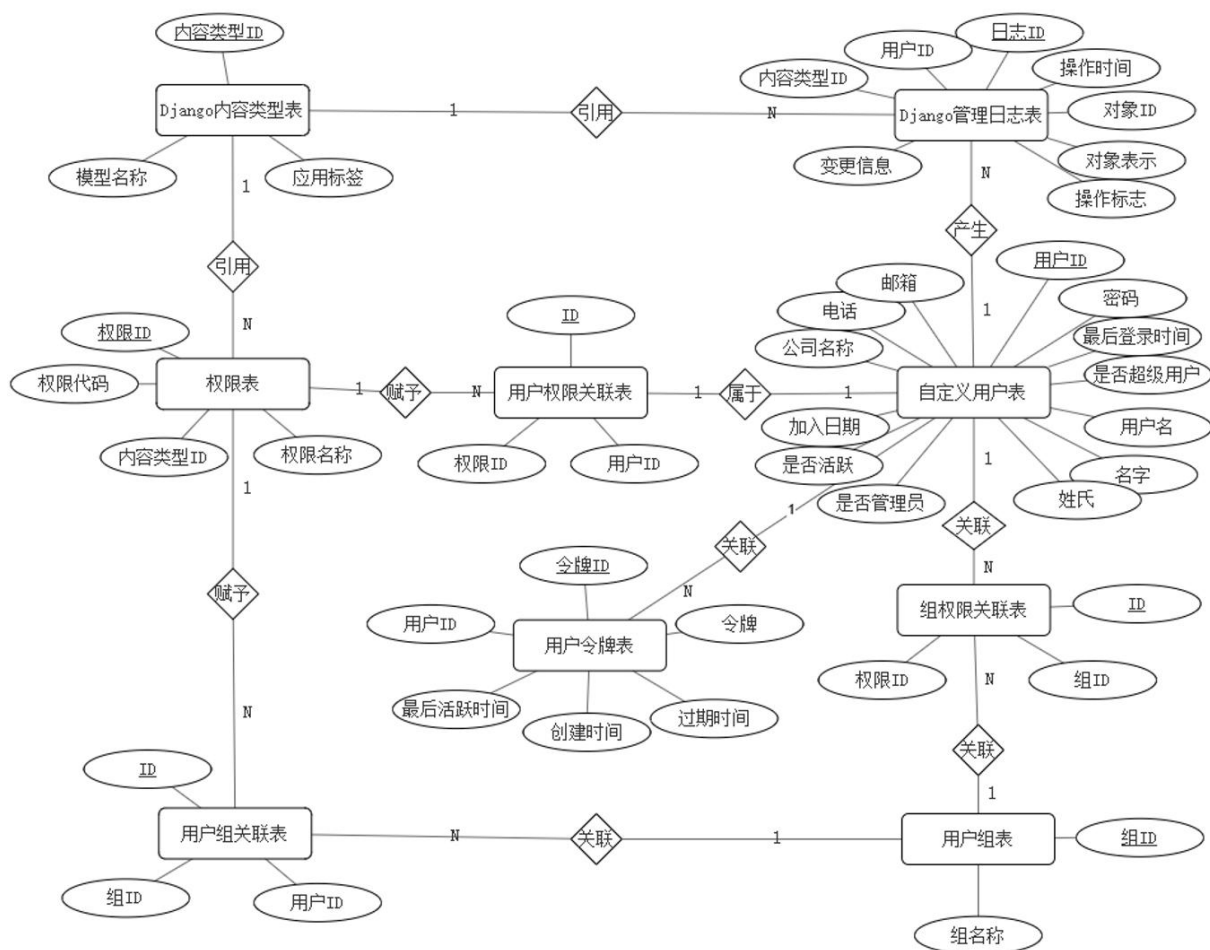


图 4.3 用户权限管理 E-R 图

该 E-R 图围绕基于 Django-admin 框架搭建的系统用户权限管理展开。在这套精心设计的权限管理机制里，用户、权限与用户组构成关键组件。用户凭借用户组关联表和用户组表构建起多对多的关联关系，即一名用户可归属于多个不同用户组；与此同时，用户能通过用户权限关联表直接获取特定权限。权限表内的权限，既可以直接授予用户，也能够经由组权限关联表赋予用户组。并且，权限借助引用 Django 内容类型表中的内容类型 ID，和具体操作对象类型挂钩，进而实现对用户各类内容类型上操作权限的精细管控。

(2)入侵检测业务核心 E-R 图

此入侵检测业务核心 E-R 图如图 4.4 所示，包含核心的告警表和规则表与 IP 封禁表及邮箱发送表进行关联。

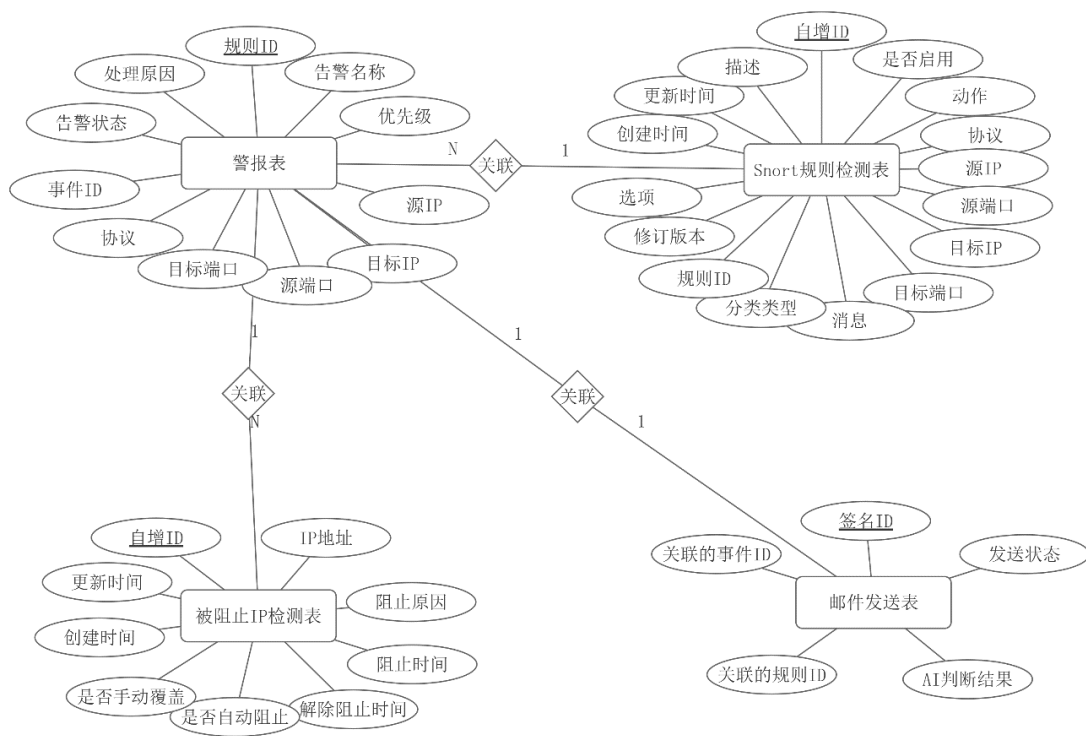


图 4.4 入侵检测业务核心 E-R 图

此 E-R 图聚焦入侵检测业务核心流程搭建。用户在入侵检测业务里处于关键地位，与封禁 IP、入侵事件、Snort 规则、告警事件等实体紧密关联。它如同核心枢纽，串联起入侵事件与触发规则，同时和邮件发送表相关联，用以记录告警事件对应的邮件发送详情，完整展现了从入侵事件触发、规则匹配，到告警处理及邮件通知的全业务流程。

4.4.2 数据库逻辑结构设计

在该网络入侵检测系统中，数据库选用 MySQL，它凭借高效稳定的数据存储与管理能力，有力支撑着系统的各项功能运作。主要包含以下主要数据表。

权限组信息表（auth_group）是实现精细化权限管理的核心组件。该表主要用于对系统内的各类权限进行分类整合与集中管理，其具体字段详情见表 4.1。

表 4.1 权限组信息表

字段名	数据类型	备注
id	INT	权限组 ID，主键，自增长
name	VARCHAR(80)	权限组名称，唯一

用户信息表（detection_customuser）作为系统的核心基础表之一，为用户认证、权限



管理、日志审计等功能提供底层数据支撑，是系统角色的核心部分。具体内容如 表 4.2 所示。

表 4.2 用户信息表

字段名	数据类型	备注
id	INT	用户 ID，主键，自增长
password	VARCHAR(128)	密码
last_login	DATETIME	上次登录时间
is_superuser	TINYINT (0/1)	是否为超级用户
username	VARCHAR(150)	用户名，唯一
first_name	VARCHAR(30)	名字
last_name	VARCHAR(150)	姓氏
is_staff	TINYINT (0/1)	是否为员工
is_active	TINYINT (0/1)	是否激活
date_joined	DATETIME	注册时间
company_name	VARCHAR(255)	公司名称
email	VARCHAR(255)	邮箱，唯一
phone	VARCHAR(20)	电话号码，唯一

权限组与权限关联关系表（auth_group_permissions）。通过该表，系统能够精准地将权限组与具体权限一一对应，使每个权限组都拥有明确的权限集合。为基于角色的权限控制机制提供了坚实的数据支撑，其具体结构与字段详情详见表 4.3。

表 4.3 权限组与权限关联关系表

字段名	数据类型	备注
id	INT	关联 ID，主键，自增长
group_id	INT	关联 auth_group.id
permission_id	INT	auth_permission.id

权限信息表（auth_permission）是系统权限控制的基础数据表，用于存储细粒度权限定义。表中包含权限名称、所属模型及权限代码名等核心字段通过字段间关联实现对用户访问和操作的精细化管控，确保系统功能按预设规则安全运行。具体内容如表 4.4 所示。



表 4.4 权限信息表

字段名	数据类型	备注
id	INT	权限 ID，主键，自增长
name	VARCHAR(255)	权限名称
content_type_id	INT	外键，关联 django_content_type.id
codename	VARCHAR(100)	权限代码名

内容类型表（django_content_type），该表用于存储 Django 中各个模型的内容类型信息，是权限管理和日志记录的关键参考表。具体内容如表 4.5 所示。

表 4.5 内容类型表

字段名	数据类型	备注
id	INT	内容类型 ID，主键，自增
app_label	VARCHAR(100)	应用标签
model	VARCHAR(100)	模型名称

管理员操作日志表（django_admin_log）作为系统审计的核心数据表，完整留存了管理员在后台的每一步操作痕迹。其具体字段设置详见表 4.6。

表 4.6 管理员操作日志表

字段名	数据类型	备注
id	INT	日志 ID，主键，自增
action_time	DATETIME	操作时间
object_id	TEXT	操作对象 ID
object_repr	VARCHAR(200)	操作对象表示
action_flag	SMALLINT	操作标志
change_message	TEXT	变更消息
content_type_id	INT	django_content_type.id
user_id	INT	detection_customuser.id

邮件发送信息表（sendemail）。用于完整记录系统每一次邮件发送操作的关键信息，



涵盖与告警事件关联的唯一标识、邮件传输过程中的状态反馈，以及通过大模型对发送内容或异常情况进行分析得出的综合性研判结论，为系统追溯邮件发送流程、评估发送效果及优化告警通知机制提供详细的数据支撑，具体字段详情见表 4.7。

表 4.7 邮件发送信息表

字段名	数据类型	备注
sig_id	VARCHAR(50)	信号 ID，可关联告警事件
sent_status	VARCHAR(50)	发送状态
AI_judgment	TEXT	大模型研判结果

用户与权限关联关系表（`detection_customuser_user_permissions`）是系统实现精细化权限管理的核心数据载体，构建起用户身份标识与操作权限集合的多维度映射链路，实现用户角色与可执行功能的动态绑定与灵活配置。具体内容如表 4.8 所示。

表 4.8 用户与权限关联关系表

字段名	数据类型	备注
id	INT	关联 ID，主键，自增长
customuser_id	INT	<code>detection_customuser.id</code>
permission_id	INT	<code>auth_permission.id</code>

用户与用户组关联关系表（`detection_customuser_groups`），表用于记录用户与权限组之间的关系，支持为用户分配多个权限组，便于用户权限的管理和关联。具体内容如表 4.9 所示。

表 4.9 用户与用户组关联关系表

字段名	数据类型	备注
id	INT	关联 ID，主键，自增长
customuser_id	INT	<code>detection_customuser.id</code>
group_id	INT	<code>auth_group.id</code>

封禁 IP 表（`detection_blockedip`）是入侵防御机制的关键组成部分，基于 `iptables` 帮助记录被系统封禁的 IP 地址及关联信息。该表通过持续更新威胁 IP 列表，为实时流量过滤提供数据依据，有效阻断异常访问，提升系统主动防御能力，具体内容如表 4.10 所



示。

表 4.10 封禁 IP 表

字段名	数据类型	备注
id	INT	封禁记录 ID，主键，自增
ip_address	VARCHAR(45)	IP 地址
block_reason	TEXT	封禁原因
block_time	DATETIME	封禁时间
unblock_time	DATETIME	解封时间
auto_blocked	TINYINT	是否自动封禁 (0/1)
manual_override	TINYINT	是否手动解除 (0/1)
created_at	DATETIME	创建时间
updated_at	DATETIME	更新时间

告警信息表（alert_table）在入侵检测系统中占据关键地位，作为入侵检测与响应模块的核心数据载体，它全面记录了系统运行过程中触发的各类告警信息。便于快速识别告警类型，其具体结构与字段详情详见表 4.11。

表 4.11 告警信息表

字段名	数据类型	备注
sid	INT	告警 ID，通常与规则 ID 对应
alert_name	VARCHAR(50)	告警名称
priority	VARCHAR(50)	优先级
src_ip	VARCHAR(50)	源 IP 地址
dst_ip	VARCHAR(50)	目标 IP 地址
src_port	VARCHAR(50)	源端口（可为空）
dst_port	VARCHAR(50)	目标端口（可为空）
protocol	VARCHAR(50)	协议类型
cid	INT	关联 ID，唯一标识 id

Snort 规则表（detection_snortrule），完整承载了 Snort 检测引擎运行所需的规则体



系。其具体字段及功能详情详见表 4.12。

表 4.12 Snort 规则表

字段名	数据类型	备注
id	INT	规则 ID，主键，自增长
enabled	TINYINT	是否启用 (0/1)
action	VARCHAR(10)	动作
protocol	VARCHAR(10)	协议
src_ip	VARCHAR(45)	源 IP 地址
src_port	VARCHAR(20)	源端口
dst_ip	VARCHAR(45)	目标 IP 地址
dst_port	VARCHAR(20)	目标端口
msg	VARCHAR(255)	消息
classtype	VARCHAR(50)	分类类型
sid	VARCHAR(20)	规则 ID 号，唯一
rev	VARCHAR(10)	版本
options	TEXT	选项
created_at	DATETIME	创建时间
updated_at	DATETIME	更新时间
description	TEXT	描述

4.5 本章小结

本章完成网络入侵检测系统设计工作。遵循模块化与可扩展原则搭建总体架构，合理划分功能模块，精心设计数据库结构。各环节紧密衔接，为系统实现筑牢根基，后续将据此高效推进开发，保障系统稳定运行与功能拓展。



第五章 系统实现

5.1 系统开发工具

本系统采用 Django5.1.3 框架开展后端开发工作，借助 Snort2.9 完成网络流量的捕获与分析任务，同时调用 Qwen 大模型 API 实现威胁情报解析和告警研判功能。系统部署环境涵盖 Python（3.11）/Django（5.1.3）开发环境、Snort2.9 部署环境、QQ 的 SMTP 邮箱服务以及千问大模型 API 服务环境。

5.2 威胁情报处理模块的实现

威胁情报处理模块实现的流程图和情报解析和规则生成的流程图如图 5.1 和 5.2 所示。

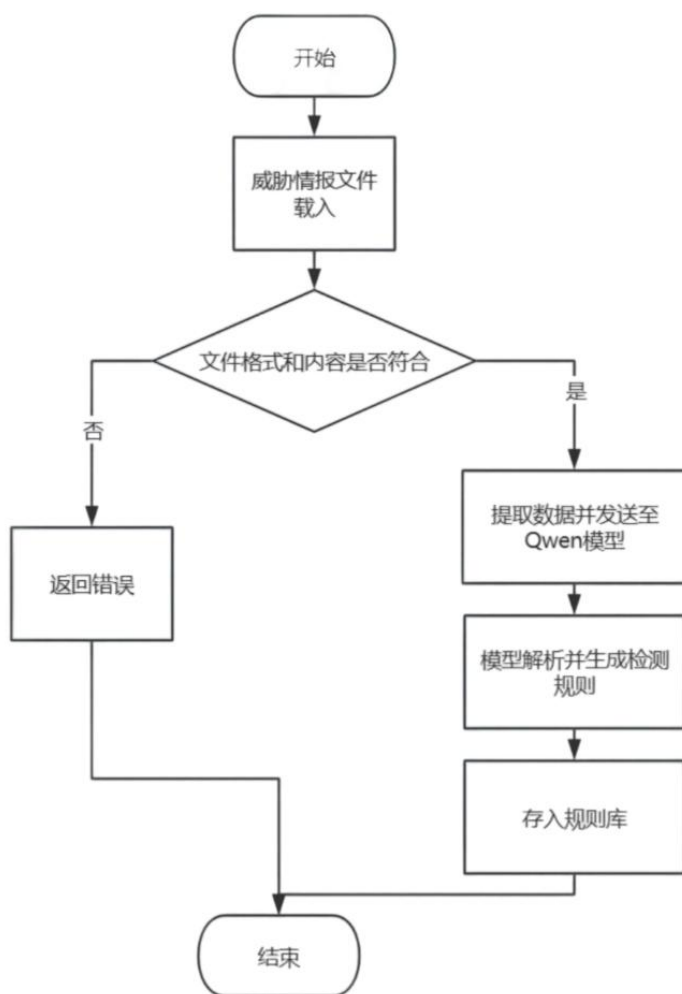


图 5.1 威胁情报处理模块流程图

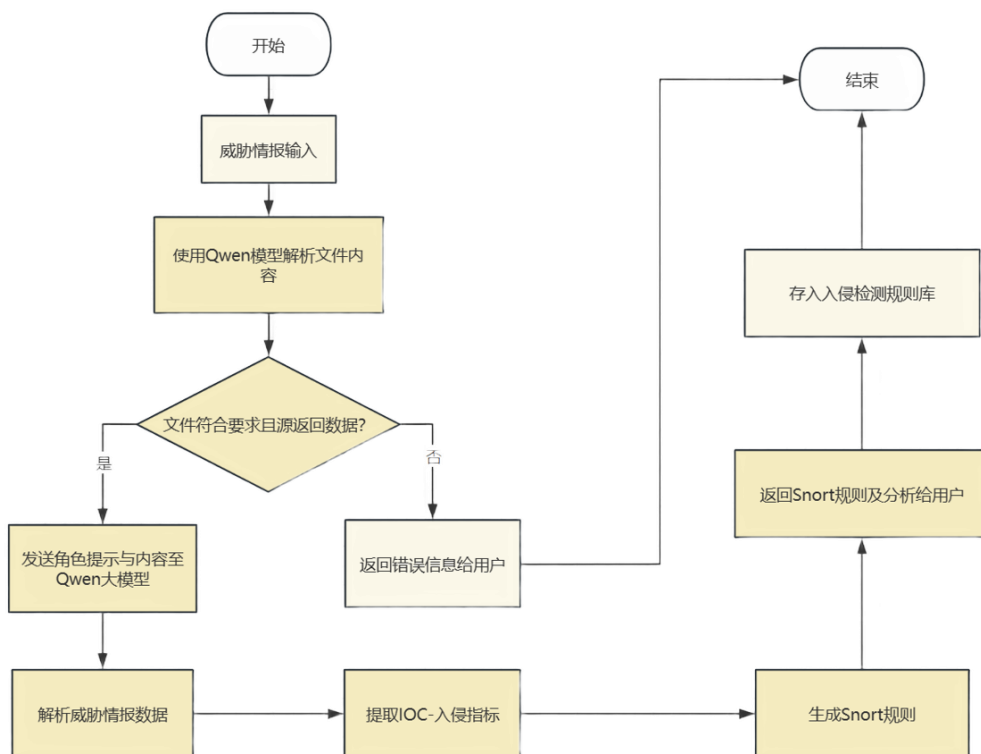


图 5.2 情报解析和规则生成流程图

通过调用通义千问 API，系统能够实现对威胁情报的自动解析和规则生成。具体实现过程中，系统首先从外部源获取威胁情报数据，然后调用通义千问 API 进行解析，提取关键信息并生成相应的 Snort 规则。生成的规则将被存入入侵检测规则库，供后续流量监控和告警分析使用。最终实现效果如图 5.3 所示。

文本输入威胁情报

生成规则

通过威胁情报文件生成规则

选择文件

HVV2023-漏洞情报-艾科思（鑫智科技）应用接入系统存在任意文件读取漏洞.pdf

上传并生成规则

通过情报订阅配置生成规则

情报URL配置地址

情报获取数量

生成规则

根据威胁情报生成的 Snort 规则:

```
alert tcp any any -> any any (msg:"高危 - 艾科思应用接入系统任意文件读取漏洞检测"; flow:established,to_server; urlen:1; content:'/.%5c.%5c.%5c.%5c.%5c.%5c.%5c.%5c/windows/win.ini'; http.uri:metadata:policy security=ips drop, service http; reference:url,HVV2023-漏洞情报-艾科思《鑫智科技》应用接入系统存在任意文件读取漏洞.pdf; sid:2025051101; rev:1;)
```

分析：规则针对特定的HTTP请求路径，有效检测利用该漏洞的尝试，减少误报可能性。

导入规则库

根据情报URL配置生成的 Snort 规则:

等待生成规则...

导入规则库

图 5.3 大模型生成规则实现效果图

5.3 流量监控与告警模块的实现

流量监控与告警模块通过部署 Snort 规则并清洗告警数据库实现流量监控与告警功能。具体实现步骤如图 5.4 所示。

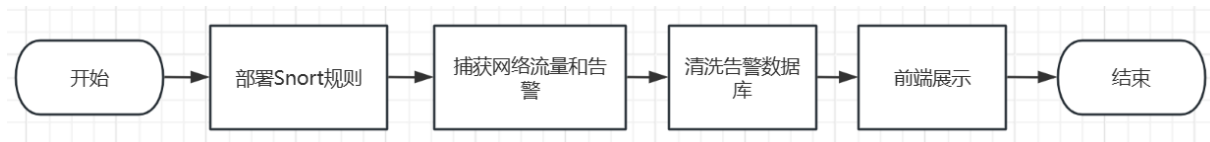


图 5.4 规则部署和告警数据库清洗流程图

(1)部署 Snort 规则：将生成的入侵检测规则部署到 Snort 流量监控服务器中，具体的路径为/etc/snort/rules/local.rules 下，以便对网络流量进行实时监控和分析处理。

(2)捕获网络流量与告警：Snort 流量监控工具捕获网络流量告警数据并将其发送给 Django 后端服务进行进一步处理。

(3)清洗告警数据库：对接收到的告警数据进行清洗和处理以便去除重复和无效的告警信息。

(4)前端展示：将清洗后的告警数据传输至前端界面进行可视化展示同时将产生的告警发送给安全管理员，供安全人员查看和操作。告警前端展示实现效果如图 5.5 所示，邮箱通知如图 5.6 所示。

搜索事件ID,传感器ID

告警时间

告警时间

告警读取状态

搜索

所有日期

2025年4月

2025年5月

2025年6月

删除

选中了 40 个中的 0 个

搜索

刷新

重置

☐	告警时间	告警名称--事件ID	优先	源IP	目标IP	源端口	目标端口	协议	告警读取状态	大模型研判	处置
☐	2025年6月4日 11:50	ICMP请求响应--87	低	192.168.43.254	192.168.43.212	未知	未知	ICMP	告警未读	告警详情	封禁IP
☐	2025年6月4日 11:50	ICMP请求响应--86	低	192.168.43.254	192.168.43.212	未知	未知	ICMP	告警未读	告警详情	封禁IP
☐	2025年6月2日 18:57	ICMP请求响应--85	低	192.168.43.254	192.168.43.212	未知	未知	ICMP	告警未读	告警详情	封禁IP
☐	2025年6月2日 18:57	ICMP请求响应--84	低	192.168.43.254	192.168.43.212	未知	未知	ICMP	告警未读	告警详情	封禁IP
☐	2025年6月1日 21:12	ICMP请求响应--83	低	192.168.43.254	192.168.43.212	未知	未知	ICMP	告警未读	告警详情	封禁IP
☐	2025年6月1日 21:12	ICMP请求响应--82	低	192.168.43.254	192.168.43.212	未知	未知	ICMP	告警未读	告警详情	封禁IP
☐	2025年5月29日 18:17	ICMP请求响应--81	低	192.168.43.254	192.168.43.212	未知	未知	ICMP	告警未读	告警详情	封禁IP
☐	2025年5月29日 18:17	ICMP请求响应--80	低	192.168.43.254	192.168.43.212	未知	未知	ICMP	告警未读	告警详情	封禁IP
☐	2025年5月28日 16:57	ICMP请求响应--79	低	192.168.43.254	192.168.43.212	未知	未知	ICMP	告警未读	告警详情	封禁IP
☐	2025年5月28日 16:57	ICMP请求响应--78	低	192.168.43.254	192.168.43.212	未知	未知	ICMP	告警未读	告警详情	封禁IP
☐	2025年5月23日 08:28	ICMP请求响应--77	低	192.168.43.254	192.168.43.212	未知	未知	ICMP	告警已读	告警详情	封禁IP
☐	2025年5月23日 08:28	ICMP请求响应--76	低	192.168.43.254	192.168.43.212	未知	未知	ICMP	告警未读	告警详情	封禁IP
☐	2025年5月22日 18:05	ICMP请求响应--75	低	192.168.43.254	192.168.43.212	未知	未知	ICMP	告警未读	告警详情	封禁IP
☐	2025年5月22日 18:05	ICMP请求响应--74	低	192.168.43.254	192.168.43.212	未知	未知	ICMP	告警未读	告警详情	封禁IP
☐	2025年5月22日 17:53	SSH爆破攻击--73	未知	192.168.43.139	192.168.43.210	40074	22	TCP	告警未读	告警详情	封禁IP
☐	2025年5月16日 12:37	Nmap TCP端口扫描攻击--32	中	192.168.43.1	192.168.43.210	16772	8089	TCP	告警未读	告警详情	封禁IP
☐	2025年5月16日 12:31	连接高危IP告警--16	中	124.221.125.254	192.168.43.210	80	47158	TCP	告警未读	告警详情	封禁IP
☐	2025年5月16日 12:31	连接高危IP告警--15	中	124.221.125.254	192.168.43.210	80	47158	TCP	告警未读	告警详情	封禁IP

图 5.5 告警数据前端展示

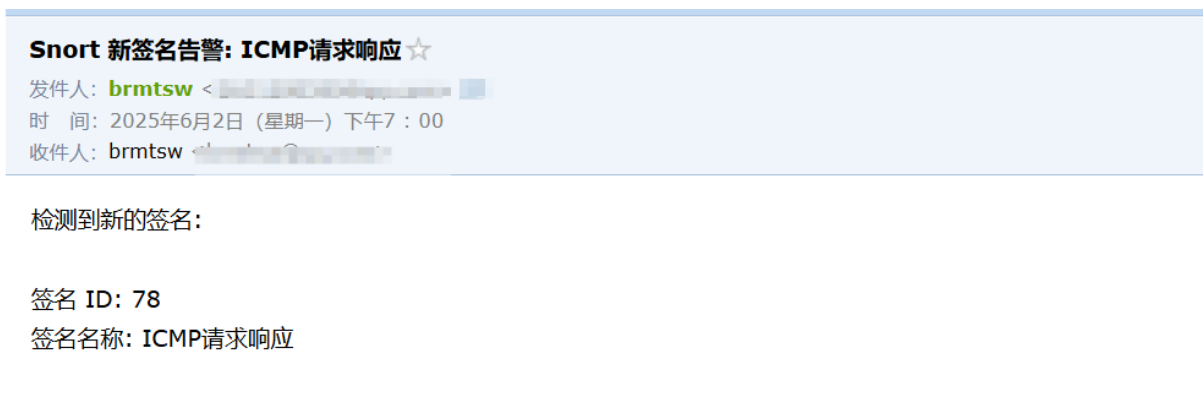


图 5.6 邮箱告警通知展示

5.4 大模型告警研判模块的实现

5.4.1 告警数据提取与大模型分析流程

大模型告警研判模块通过提取告警数据并利用大模型进行分析处理实现高效告警研判功能^[17]。具体实现步骤如图 5.7 所示。



图 5.7 告警数据提取和大模型分析流程图

- (1)提取告警数据：从告警数据库中提取待研判的告警数据和触发这条告警的规则。
- (2)数据预处理：在将提取到的告警数据发送给大模型 API 之前，通常需要进行数据预处理。包括格式转换（确保数据格式与大模型 API 的要求一致）。数据预处理能够显著提升大模型的分析效率和准确性。
- (3)大模型分析处理：qwen 大模型接收到告警数据后，将自动执行事件分类、关联分析、趋势预测及威胁等级评估等任务。基于大模型强大的语义理解与推理能力，能够更精准地识别攻击意图，判断是否为误报，并输出研判结果。同时，模型还将结合历史数据与当前上下文信息，提供短期处置建议与长期安全加固方向，辅助安全人员快速响应与决策。
- (4)结果后处理：大模型分析完成后，会返回分析结果。这些结果可能需要进行进一步的后处理，格式化输出（按照预定的模板或格式输出结果）。
- (5)反馈与优化：根据大模型的分析结果和实际应用情况，对大模型告警研判结果和对应的触发规则进行反馈和优化。能够使得大模型告警研判模块更加适应不断变化的安

全威胁环境和降低原始规则的误报率。

5.4.2 研判结果展示与误报优化

(1)研判结果展示

为了提升安全人员对告警事件的理解效率与响应速度，系统提供结构化、可视化的研判结果展示界面：

可视化展示：设计直观的前端界面，用于呈现大模型分析后的告警研判报告。展示内容包括：告警命中规则详情（如规则 ID、规则内容、查询时间）。研判结论（是否为误报、攻击意图、威胁等级）。处置建议（短期应对措施与长期加固策略）。相关证据数据（提取关键上下文信息，辅助判断）。规则反哺建议（是否需要调整检测规则及其原因）。研判结果展示与误报优化实现效果如图 5.8 所示。



图 5.8 研判结果展示与误报优化效果实现图

(2)误报优化

尽管当前系统尚未引入人工反馈机制，但通过大模型的智能研判能力，已在一定程度上减少了误报影响。基于 Qwen 大模型对告警上下文的理解能力，自动判断告警是否为误报，结合规则语义、流量行为、历史模式等多维度信息进行综合评估。每次研判完成后，模型会输出是否建议修改当前告警规则，并说明理由。研判结果与规则建议将通过 LogEntry 模块记录到系统日志中，供后续分析使用，为未来规则调优提供依据。

5.5 黑名单反制功能实现

基于 iptables 的自动化 IP 封禁与解封机制，构建了从威胁检测到网络层阻断的主动防御体系。核心实现流程和实现效果如下图 5.9 和 5.10 所示。

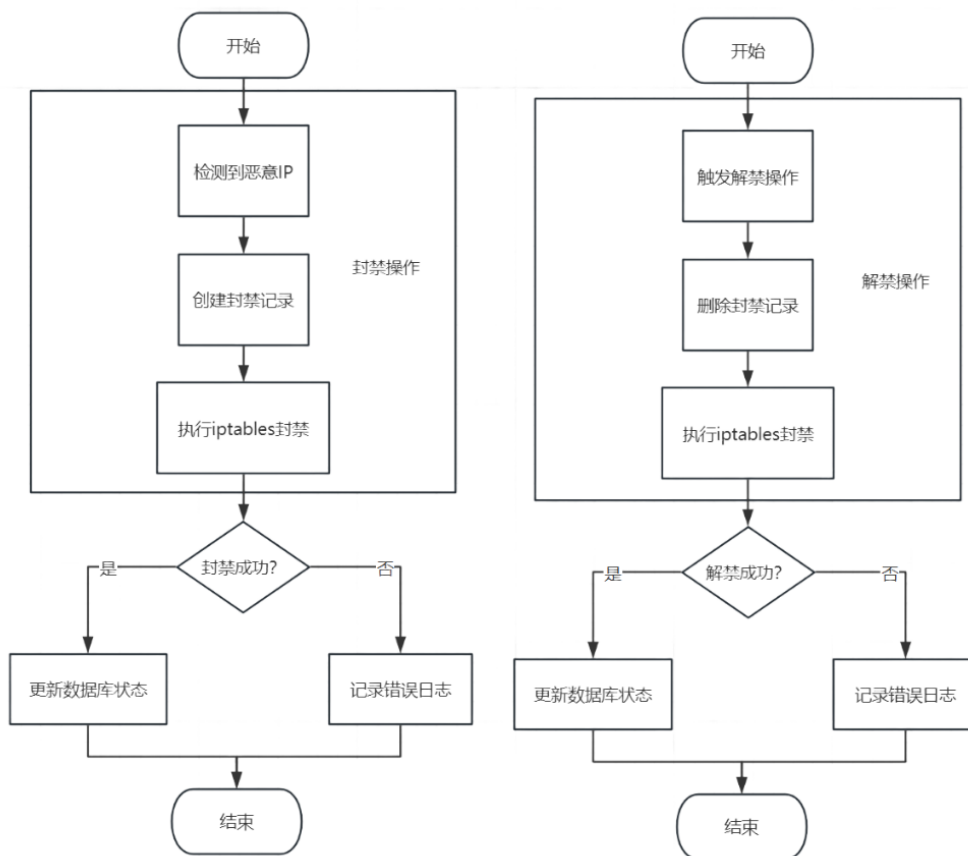
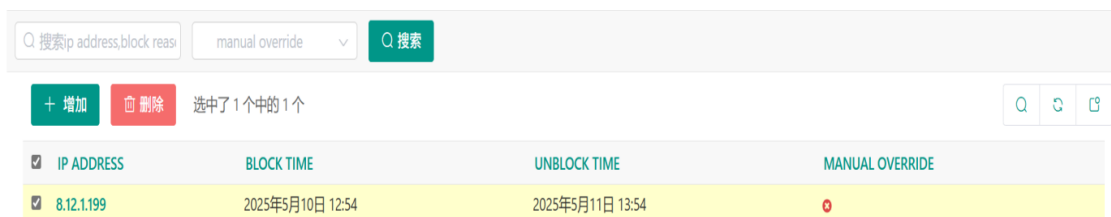


图 5.9 黑名单反制功能实现流程图



IP ADDRESS	BLOCK TIME	UNBLOCK TIME	MANUAL OVERRIDE
8.12.1.199	2025年5月10日 12:54	2025年5月11日 13:54	0

图 5.10 黑名单反制功能实现效果图

(1) 自动化封禁与解封流程

系统通过 SSH 远程执行 `iptables -I INPUT -s {ip} -j DROP` 插入阻断规则，并计算解封时间差值作为封禁时长。删除记录时，优先执行 `iptables -D INPUT -s {ip} -j DROP` 删除规则，即使远程操作失败，仍强制更新本地数据库标记为已解封状态，确保状态最终



一致性。

(2)远程命令执行与安全管控

采用 Paramiko 库建立加密 SSH 通道，通过 Django 配置中心管理防火墙服务器凭证。执行命令时使用专用低权限账号，并限制 sudo 权限仅允许/sbin/iptables 操作。封禁规则插入 INPUT 链顶端保证优先匹配，同时通过 iptables-save 持久化存储规则，避免系统重启后失效。

(3)防御时效与容错机制

用户指定解封时间基础上增加 1 小时缓冲时间，防止时钟不同步导致的提前解封。批量解封操作通过重写 delete_queryset 方法实现逐条 IP 解封，避免原子性操作风险。引入连接池技术复用 SSH 会话，结合异步任务队列降低高并发场景下的系统延迟。

5.6 本章小结

本章详细介绍了大模型告警研判模块的实现过程，包括告警数据的提取与大模型分析流程，以及研判结果的展示与误报优化。通过集成大模型技术，大模型告警研判模块能够实现对告警数据的高效、准确研判，同时使用黑名单反制功能，减少恶意攻击者对企业的危害，为网络安全运营提供有力支持。



第六章 系统测试与分析

6.1 系统测试环境

系统测试环境配置信息如表 6.1 所示。

表 6.1 测试环境配置信息表

测试环境	名称	版本号或大小
硬件环境	CPU 处理器	13th Gen Intel(R) Core(TM) i9-13900HX
	硬盘	1TB
	机带 RAM	32.0 GB
软件环境	操作系统	Windows11、Centos7
		(VMware18)
	开发语言	Python3.11
	数据库	MySQL8.0
	入侵检测引擎	Snort2.9

此配置环境模拟了实际部署场景，确保测试结果能够真实反映系统性能。

6.2 测试数据集来源与构成

本次测试使用的数据集覆盖多类型、多场景的威胁情报，均来源于公开权威平台及实际业务场景采集，确保样本的真实性和多样性。具体构成如下。

(1)恶意 IP 数据集

来源：公共威胁情报平台（如微步在线 X 情报社区、奇安信威胁情报中心等）。
历史攻击日志中提取的活跃攻击 IP。

样本特征：包含 100 个样本，覆盖常见攻击类型（如 DDoS 攻击源、暴力破解 IP、C2 端 IP）。

(2)恶意域名数据集

来源：域名黑名单数据库（DNSBL）。
样本特征：包含 50 个样本，包括仿冒知名网站域名、僵尸网络域名。

(3)钓鱼 URL 数据集

来源：公开钓鱼网站数据库（PhishTank）。
样本特征：包含 30 个样本，覆盖常见钓鱼场景（账号窃取、虚假中奖等）。



(4)病毒特征码数据集

来源：木马病毒特征库（360 和微步情报在线）公开的特征码片段。

样本特征：包含 8 个样本，涵盖常见病毒类型（勒索软件、木马、蠕虫等）。

6.3 功能测试

6.3.1 生成规则有效性测试

本次测试目的主要是验证本系统根据威胁情报生成的 Snort 规则的有效性。测试方式为使用不同类型威胁情报的测试用例，如恶意 IP、域名、URL、病毒特征码等，通过批量请求脚本进行测试。检查规则是否能够准确地识别出对应的威胁和对误报的规则进行反馈。测试用例如表 6.2 所示。

表 6.2 测试用例表

测试用例	预期结果	测试结果
测试包含内网 IP 和 https 白名单域名的情报	反馈用户去除该情报产生的规则	符合预期
包含多种特征的威胁情报文件	系统准确识别且生成的规则不丢失关键 IOC	符合预期

测试规则准确性结果情况（均为系统生成的规则）如表 6.3 所示。

表 6.3 系统检测结果

测试对象	样本数量(个)	TP	FN
恶意 IP	100	98	2
恶意域名	50	47	3
钓鱼 URL	30	30	0
病毒特征码	8	7	1

测试结果表明，系统在生成 Snort 规则在检测攻击方面具有较高的准确性和较低的漏报率，能够有效识别多种类型的威胁情报。

以 ping 攻击检测为例，双方规则对比如图 6.1 所示（右为本系统生成规则），在可读性上，以自然语言呈现，如“高危-异常 Ping 流量检测”，一目了然，远胜 Snort 原规则单纯技术术语描述。场景性方面，针对 Ping 攻击设定“60 秒内双向达 100 次”这样精准阈值，贴合实际攻击场景，而原规则无相关设置，易误报漏报。



```
1 alert icmp any any -> any any (msg:"ICMP Echo Request"; itype:8;  
  classtype:icmp-event; sid:1000001; rev:2;)
```

```
1 alert icmp any any -> any any (msg:"高危 - 异常Ping流量检测";  
  icmp_type:8; threshold: type both, count 100, seconds 60;  
  classtype:icmp-event; sid:2025060101; rev:1;)  
2 分析：通过限制icmp类型为请求且设置阈值减少误报 可能存在正常高频ping场景需微  
  调
```

图 6.1 Ping 攻击检测规则生成对比图

通过对比发现系统大模型生成规则优势显著。本系统生成的规则更适应复杂网络环境，实用性与精准度更高。

6.3.2 告警研判响应时间测试

为评估系统在不同告警数量下的 AI 判研响应效率，进行了多轮压力测试。通过模拟不同规模的告警输入，记录从告警产生到 AI 完成研判的总耗时，并计算平均响应时间，结果如表 6.4 所示。

表 6.4 告警响应和 AI 研判平均响应时间表

告警数量(次)	告警平均生成时间(ms)	AI 研判响应时间(s)
60	96	17.329
80	103	18.121
100	108	19.425

随着告警数量增加，AI 判研总耗时呈增长趋势，但单个告警的平均生成时间保持稳定（96 至 108 ms），表明系统具备良好的可扩展性和稳定的实时处理性能。

6.3.3 系统基础功能测试

为全面验证系统核心功能的可用性与稳定性，针对系统的基础功能开展专项测试，覆盖系统全业务流程。测试通过模拟真实操作场景，验证各功能是否符合设计预期。具体测试内容及结果如表 6.5 所示

表 6.5 系统基础功能测试表

测试功能	预期效果	测试结果
登录登出功能	登录流程安全顺畅，登出清除会话	符合预期
用户/组添加和删除	管理员可创建用户组、分配权限	符合预期
用户信息修改功能	用户可修改个人资料	符合预期
告警生成和查看功能	告警生成和查看完整无丢失	符合预期



表 6.5（续表）系统基础功能测试表

测试功能	预期效果	测试结果
规则生成、添加和删除功能	规则生成、添加和删除功能正常，规则版本可回溯。	符合预期
IP 封禁功能	对确认的攻击源 IP 一键封禁，支持设置封禁时长或手动解封。	符合预期
告警信息可视化功能	通过仪表盘动态展示告警趋势、威胁类型分布等数据	符合预期
日志记录和查看功能	自动记录用户操作、系统事件及告警处理日志，支持关键字检索	符合预期

经测试系统基础功能均通过测试验证，各模块操作逻辑清晰、响应及时，权限管理严谨，日志追溯完整，符合业务场景下的功能需求。

6.4 性能测试

系统在空闲与峰值负载下的资源占用对比如表 6.6。

表 6.6 系统资源占用对比表

状态	CPU 占用率(%)	内存消耗(MB)	线程数(个)
空闲	1.2	278.1	15
峰值	21.3	324.8	23

测试结果表明，系统在空闲状态下资源占用较低，在峰值负载下资源占用有所增加，但仍保持在合理范围内，不会对系统稳定性造成影响。无论是数据处理能力、长时间稳定性，还是网络环境适应性，该系统均表现优异，完全能够满足实际业务场景中的高性能需求。

6.5 本章小结

本章通过对网络入侵检测系统进行全面的功能测试和性能测试，验证了系统各项功能的准确性和有效性，以及在不同负载条件下的稳定性和资源占用情况。测试结果表明，本系统能够满足设计要求，为网络安全提供有力保障。



第七章 总结与展望

7.1 总结

(1)随着互联网技术的迅猛发展和网络攻击手段的日益复杂，网络入侵检测系统在保障网络安全方面的重要性愈发凸显。针对传统 IDS 存在的规则更新滞后、误报率高等核心问题，本文提出并实现了一种基于大语言模型与威胁情报的网络入侵检测系统，通过融合大模型技术与传统工具 Snort，构建了“自动化规则生成—实时流量检测—智能告警研判—动态反制响应”的完整防护链路，显著提升了网络入侵检测的准确性与实时性 [18]。

(2)在系统架构设计层面，本文遵循模块化与可扩展性原则，基于 Django 框架构建分层体系：利用 Django 的 ORM 功能实现数据库高效操作，通过 SimpleUI 增强管理界面的交互体验，确保前端展示与后端逻辑的解耦。在威胁情报处理模块，系统通过调用大模型 API，将非结构化威胁情报（如漏洞报告、攻击技术文档）自动解析为恶意 IP、攻击载荷等入侵指标，并按照 Snort 规则语法生成检测规则，显著减少人工干预的同时，提升规则生成的时效性与精准度。

(3)流量监控与告警分析作为系统核心功能，通过 Snort 实时捕获网络流量并执行规则匹配，一旦检测到异常行为即触发告警。在此基础上，大模型对告警数据进行多维度语义分析，结合攻击时序、资产脆弱性等上下文信息实现误报过滤与威胁分级，最终通过可视化模块以图表、列表等形式呈现告警详情，辅助管理员快速定位风险并制定响应策略。

(4)为强化主动防御能力，系统集成基于 iptables 的自动化 IP 封禁机制，对确认的恶意 IP 实施快速阻断，并记录封禁日志以支持后续溯源分析。这一功能实现了从威胁检测到防御响应的自动化闭环，有效缩短攻击处置时间，提升系统整体安全性。

(5)通过系统实现与测试验证，本文提出的方案在规则生成准确性、告警响应速度、资源占用等关键指标上表现优异，满足企业级网络安全防护的实际需求。研究成果不仅为入侵检测领域提供了“AI 技术 + 传统安全工具”的融合思路，更通过工程化实践为金融、政务等对安全性要求严苛的场景提供了可落地的解决方案，助力构建智能化、高效化的网络安全防御体系。



7.2 ~~展望~~

尽管本文在网络入侵检测系统方面取得了一定的研究成果，但仍有许多方面需要进一步探索和完善。未来研究可着眼于多源威胁情报融合，鉴于网络安全威胁日益复杂化，单一威胁情报源已难以满足实际需求，因此研究如何融合来自多个威胁情报源的数据，提高威胁情报的全面性和准确性，进而生成更加有效的入侵检测规则，将成为重要方向。同时，深度学习与强化学习在图像识别、自然语言处理等领域已取得显著成果，将这些技术进一步探索应用于网络入侵检测中，通过不断学习和优化，可提升系统的自适应能力和智能化水平^[19]。此外，随着云计算技术的普及，云化部署已成为系统发展的重要趋势，研究系统的跨平台兼容性，实现不同操作系统和硬件环境下的稳定运行，并将系统部署到云端，利用云计算资源提高系统的可扩展性和弹性，也是未来的研究重点^[20]。在处理威胁情报和用户数据时，隐私保护和合规性是不可忽视的重要问题，未来需更加注重系统的隐私保护能力，确保在处理敏感信息时符合相关法律法规的要求。最后，良好的用户交互界面和可视化展示方式对于提高系统的易用性和管理效率至关重要，未来可进一步优化用户交互界面和可视化展示方式，提供更加直观、便捷的操作体验，以帮助安全人员更快速地理解和响应网络安全事件。



参考文献

- [1] 符能. 基于网络安全大模型的网络安全防护技术研究 [J]. 数字通信世界, 2025, (04): 31-33.
- [2] 颜慧芳. 论电信运营商网络安全防护体系中威胁情报的应用[J]. 中国宽带, 2024, 20(06): 49-51. DOI:10.20167/j.cnki.ISSN1673-7911.2024.06.17.
- [3] Arun A, Nair A S, Sreedevi A G. Zero day attack detection and simulation through deep learning techniques[C]//2024 14th International Conference on Cloud Computing, Data Science & Engineering (Confluence). IEEE, 2024: 852-857.
- [4] 郑承蔚, 王海凤, 刘瑞. SDN 中 DDoS 攻击检测研究综述[J]. Journal of Computer Engineering & Applications, 2024, 60(24): 5-6.
- [5] Tao Y, Zhang Y, Ma S, et al. Combining the big data analysis and the threat intelligence technologies for the classified protection model[J]. Cluster Computing, 2017, 20: 1035-1046.
- [6] Agrawal S, Sarkar S, Aouedi O, et al. Federated learning for intrusion detection system: Concepts, challenges and future directions[J]. Computer Communications, 2022, 195: 346-361.
- [7] 夏文豪, 张连成, 郭毅, 等. NDP-FD6: 一种 IPv6 网络 NDP 洪泛行为多分类检测框架[J]. Application Research of Computers/Jisuanji Yingyong Yanjiu, 2024, 41(10): 3141-3148.
- [8] 袁晓桂. 人工智能系统在通信网络入侵检测技术中的应用[J]. 信息与电脑(理论版), 2023, 35(09): 176-178.
- [9] 孟楠, 周成胜, 赵勋. 生成式人工智能赋能网络安全运营降噪能力研究[J]. 信息通信技术与政策, 2024, 50(08): 24-31.
- [10] 吴彦柱. 基于网络入侵检测系统的算法分析[J]. 电子技术与软件工程, 2013, (24): 119. DOI:10.20109/j.cnki.etse.2013.24.083.
- [11] 陆泉名. 网络入侵检测系统的设计与实验[J]. 无线互联科技, 2022, 19(14): 149-151.
- [12] 黄振昊. 网络入侵检测的设计和实现方案[J]. 科技通报, 2019, 35(12): 82-86. DOI:10.13774/j.cnki.kjtb.2019.12.015.
- [13] 李菲. 网络入侵检测系统的设计及实现分析[J]. 电脑与信息技术, 2017, 25(05): 58-60. DOI:10.19414/j.cnki.1005-1228.2017.05.015.
- [14] 杨钧. 公共网络入侵检测系统的设计[J]. 网络安全技术与应用, 2016, (06): 24-25.
- [15] 李妍. 计算机网络入侵检测系统的研究[J]. 科技风, 2019, (25): 105-106. DOI:10.19392/j.cnki.1671-7341.201925092.
- [16] 倪立. 常用网络入侵检测系统分析研究[J]. 新媒体研究, 2015, 1(06): 20-21. DOI:10.16604/j.cnki.issn2096-0360.2015.
- [17] 崔斌. 基于贝叶斯分类算法的网络入侵检测系统设计[J]. 信息与电脑(理论版), 2022, 34(13): 65-67.
- [18] 王超. “AI+网络安全”: 构建智能防御新防线——访绿盟科技集团股份有限公司副总裁陈珂[J]. 信息安全与通信保密, 2024, (09): 14-18.
- [19] Yzzogh H, Benaboud H. A comprehensive overview of machine learning for intrusion detection in software-defined networking [J]. Innovations in Systems and Software Engineering, 2025, (prepublish): 1-23.
- [20] Nie X, Xing J, Li Q, et al. Intrusion detection algorithm of wireless network based on network traffic anomaly analysis [J]. Egyptian Informatics Journal, 2025, 30 100689-100689.



致谢

时光飞逝，本科四年如白驹过隙，转瞬即逝。回溯过往，我与信息安全专业的缘分始于一场偶然的专业分流，那时的我，面对这个神秘又陌生的领域，满是不解与迷茫。代码的晦涩、知识的繁杂，都曾让我在探索的道路上踉跄前行。

大学四年我特别感激的就是我现在的指导老师。他是我们心中的良师益友。一直秉持“用心做教育，用爱做人师”的理念，用实际行动诠释着师者的担当。改论文时，他逐字批注，严谨细致。四年来的各种比赛，老师都时常和我们一起复盘。在他的悉心指导下，我参与了众多项目和比赛，也取得了一点不错的成绩，同时对信息安全专业有了更深的热爱。

回看过去四年，代码报错时的抓狂、攻克和发现漏洞后的狂喜，都成了这段时光里难忘的注脚。那些和朋友们并肩作战的日子，为了比赛全力以赴的瞬间，都成为了我大学生活中最宝贵的财富。

除了老师的引领，家人的支持也是我前进的动力。二十余载，他们用无私的爱为我遮风挡雨，在我迷茫时给予鼓励，在我疲惫时送上温暖。他们的信任与陪伴，是我勇往直前的底气。

最后，再次感谢我指导老师以及所有学院老师的悉心教导，感谢家人的默默付出，感谢朋友们的并肩同行，也感谢学院提供的宝贵机会。未来，我将带着这份感恩，勇敢前行，不负青春。